Simulation of Robotic Systems Lab 06

Lätitzia Brandt

July 9, 2025

Task 1

Your task is to implement an application, where you steer the car from the start to the goal configuration. The car is seen as a rigid body, which can move freely in 3D (translation in x and y, and rotation around the z-axis), so there are no constraints due to the wheels.

You can play around with different OMPL planners (RRT, RRTConnect, RRT-star, ...). You can identify the available planners in the child script file. You can also generate different collision free motions by moving obstacles around.

Implementation

Holonomic path planning

```
goalHandle = sim.getObject('./GoalConfiguration')
initPos = sim.getObjectPosition(robotHandle,-1)
goalPos = sim.getObjectPosition(goalHandle,-1)
initRot = sim.getObjectOrientation(robotHandle,-1)
goalRot = sim.getObjectOrientation(goalHandle,-1)
t = simOMPL.createTask('t')

# free omnidirectional motion (x, y, θ) with no constraints on turning radius
ss = [simOMPL.createStateSpace('2d',simOMPL.StateSpaceType.pose2d,
    robotHandle,[-0.5,-0.5],[0.5,0.5],1)]
simOMPL.setStateSpace(t,ss)

# set algorithm here: e.g. RRTstar, TRRT
simOMPL.setAlgorithm(t,simOMPL.Algorithm.RRT)
simOMPL.setCollisionPairs(t,[sim.getObject('./StartConfiguration/Car'),
     sim.handle_all])

startPos = [initPos[0],initPos[1],initRot[2]]
endPos = [goalPos[0],goalPos[1],goalRot[2]]
simOMPL.setStartState(t,startPos)
simOMPL.setGoalState(t,endPos)
simOMPL.setup(t)
r,path = simOMPL.compute(t,4,-1,800)
print(simOMPL.hasSolution(t))

# redacting pre-written code
[...]

sim.addDrawingObjectItem(lineContainer, None)
```

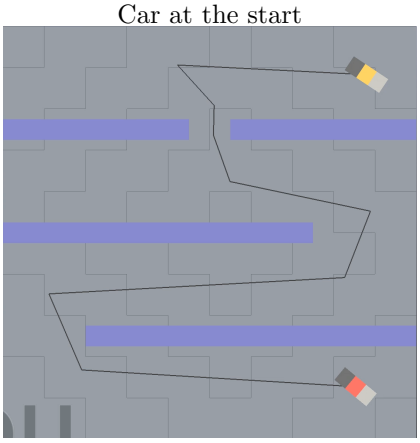
Explanation

First we save the car's initial and goal configuration to their respective variables. Then we create an OMPL task object t that will handle the path planning, and we create ss which is a component for the state space. This state space is of type 'pose2d', meaning the state includes (x, y) for position and (θ) for orientation. It also has the bounds [-0.5, -0.5] to [0.5, 0.5] which specify the min. and max. x and y coordinates that will be considered during path planning.
We then set the state space of the task object using the state space component. After that we can choose an algorithm for the task object to use. In this case we have selected 'RRT'.
Before computing the path, however, we must set the collision pairs for the task object so that OMPL knows which objects to check for collisions during path planning.
Finally, we set the start and end states (extracted from the initial and goal configurations) and set up the OMPL classes with the information contained in the task object and compute the collision-free path.
The compute function takes a maximum of 4 seconds to search a path, then simplifies it and returns true if a solution has been found, as well as the solution itself (= the states from start to end; in this case, there is a minimum of 800 states).

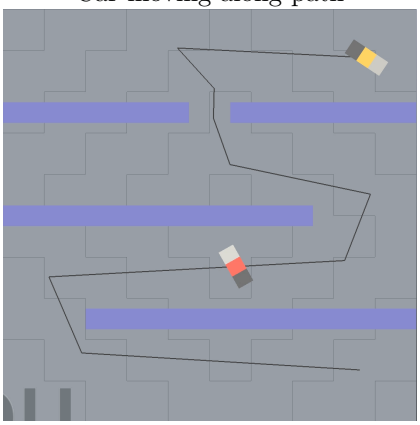
But before the car is moved along the path, we check if there exists a solution and print the boolean value.

Screenshots

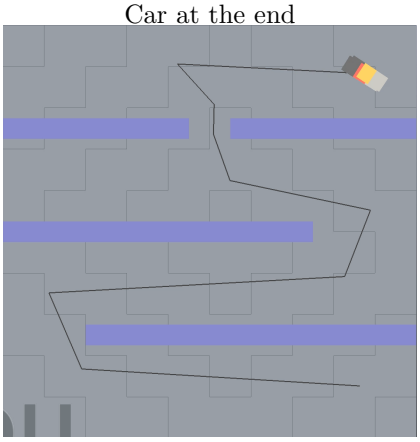
Car at the start



Car moving along path



Car at the end



Task 2

Your task is to implement an application, where you steer the car from the start to the goal configuration. The car is not anymore seen as a rigid body, which can move freely in 3D, but has to move tangentially with respect to the current steering angle.
It is recommended to use the Dubins state space, which allows 3 discrete motions straight, left and right.

```
ss=[simOMPL.createStateSpace('2d',simOMPL.StateSpaceType.dubins,
    robotHandle, [-0.5, -0.5], [0.5, 0.5], 1)]
```

You can control the steering angle left and right by the turning radius of the car (2 nd parameter).

```
# state space ss, turning radius, backward = True (allows backward motions)
simOMPL.setDubinsParams(ss[0], 0.05, True)
```

You can play around with different OMPL planners (RRT, RRTConnect, RRT-star, ...). You can identify the available planners in the child script file. You can also generate different collision free motions by moving obstacles around.

Implementation

Non-holonomic path planning

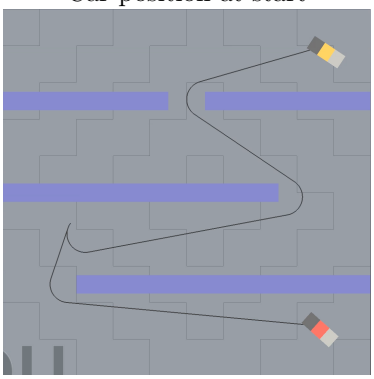
```
# dubins state space with min turning radius constraint
ss = [simOMPL.createStateSpace('2d', simOMPL.StateSpaceType.dubins,
     robotHandle, [-0.5, -0.5], [0.5, 0.5], 1)]
# provides finer collision checks along path
simOMPL.setStateValidityCheckingResolution(t, 0.002)
# state space ss, turning radius
# backward = True (allows backward motions)
simOMPL.setDubinsParams(ss[0], 0.05, True)
```

Explanation

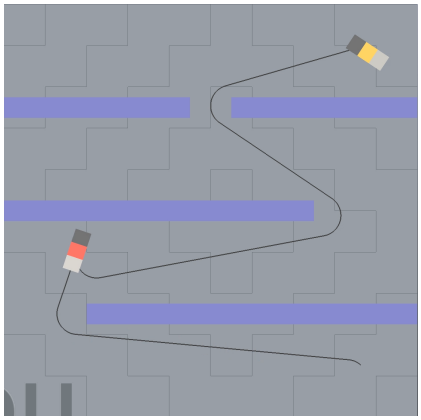
We do as we did in Task 1, setting the initial and goal configurations and so on. Here, however, we use a different state space when creating the state space component ss. It is still a 2d-type state space but now with an added constraint. Using the Dubins parameters, we set the minimum turning radius of the car to 0.05, and set backward motion to true to allow the car to drive in reverse as well.
After that, the same code follows as in Task 1.

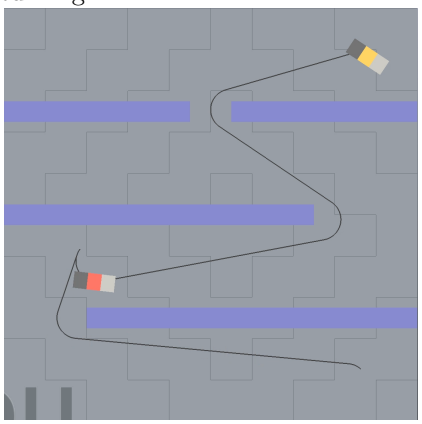
Screenshots

Car position at start



Car when turning





Car position at end

